Technologies for IoT Ecosystem – A.A. 2025/26

Hardware Laboratory

Part 2

Smart Home Controller (Local Version)

2.1 Smart Home Controller (Local Version)

Write a sketch capable of using the Arduino as a control system for "smart home". The system must manage a series of sensors and combine their readings to control a "smart" heating and air conditioning (AC) system.

Specifically, the program must implement all of the following functionalities:

a. Drive the fan to emulate an air conditioner, with speed linearly proportional to temperature, starting from a minimum set-point value (e.g. 25°C) and reaching a maximum speed corresponding to a second set-point, greater than the first (e.g. 30°C).

b. Turn on the red LED, emulating a resistive heater, with light intensity proportional to temperature, starting from a maximum set-point value (e.g. 20°C) and reaching a maximum intensity corresponding to a second set-point, lower than the first (e.g. 15°C).

c. Monitor the presence of people in the room through a motion sensor based on Passive InfraRed (PIR) technology. For simplicity, assume that a single detected movement is sufficient to confirm the presence of at least one individual in the room. Subsequently, the system may assume that there are no more people in the room if no movements are detected for a time equal to the value of the constant `timeout_pir` (for example 30 min).

d. Use the microphone readings to detect the presence of individuals located in the blind zone of the PIR sensor. To avoid interpreting occasional noises (for example the passage of an ambulance) as presence, confirm presence only if the sensor detects at least `n_sound_events` sounds above a threshold `sound_threshold` within a time interval equal to `sound_interval` (e.g., 10 events in 10 minutes). Similarly to the PIR sensor, assume that there are no people in the room if no further noise events are detected for a time equal to `timeout_sound` (e.g., 60 min, and in general different from `timeout_pir`). Be careful to set the threshold on the samples measured by the microphone so that it does not detect the noise of the fan as an indication of presence of people in the room.

e. Combine the two measurements described in points 3 and 4 so that if at least one of the two sensors detects a presence, the system assumes there are people in the room.

f. Automatically adjust the 4 temperature set points (2 for air conditioning and 2 for heating) based on the presence of people in the room. In other words, the system must store two different values for each set-point. Depending on whether a person is detected, the system must use one value or the other.

g. Display all relevant information on the LCD display. In particular:
   i. The current measured temperature
   ii. The activation percentage of the fan and the heater (0-100%).
   iii. An indication of whether the system is detecting the presence of people in the room or not
   iv. The current value of the 4 set-points.

   Given the limited number of characters available on the display, it will probably be necessary to use a schematic visualisation and alternate between multiple screens (for example, every 5 seconds), as shown in Figure 1.

h. Allow updating the 4 set-points via simple commands (with a format at each group's discretion) sent from the PC to the Arduino via the serial port.

i. **Bonus:** once the first version of the program (points 1–8) is completed, remove (without deleting the code from the sketch; simply comment it out) the functionality described in point 4. Then use the green LED to emulate a "smart" light bulb. In particular, the LED can be turned on by a double clap. Turning off may occur in the same way or automatically when the system does not detect people in the room. Try (as much as possible) to prevent the lamp from turning on and off, even due to other types of noise unrelated to hand clapping.

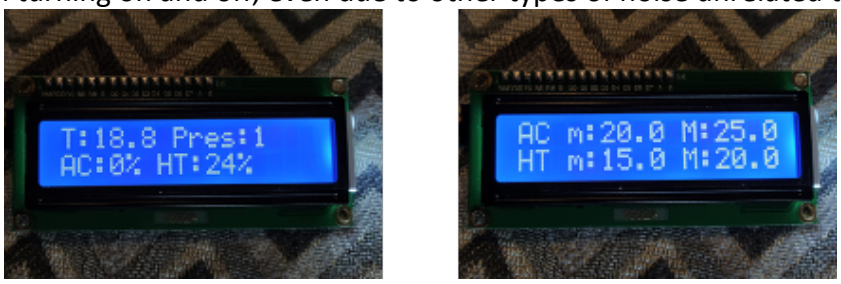

Figure 1: Example of display output for the "smart home controller". The two screens are displayed alternately at 5-second intervals. Legend: AC = air conditioner, HT = heater, m = minimum set-point, M = maximum set-point.